



Containerisation Express app

Containerizing Express App for development with Docker

Introduction

Docker has revolutionized the way we build, package, and deploy applications. By containerizing applications, we ensure consistent behavior across different environments, be it a developer's local machine , a staging environment, or production .

Problem Statement 🤔

Imagine working on an application on your local machine, and everything works perfectly. However, when you deploy it to production, things break 😞. This inconsistency between environments is often summarized by the phrase: "But it works on my machine!"

Docker addresses this problem by allowing you to package your application along with all its dependencies into a consistent environment known as a container. This container can be run uniformly across different platforms 🌐.

Objectives 🎯

By the end of this lesson, you should be able to:

1. Understand the basics of Docker.
2. Containerize an Express app 📦.
3. Run the containerized app on your local machine.

What is Docker? 🐳

Docker is a platform that enables developers to create, deploy, and run applications in containers. A container is a standardized unit of software that contains everything the software needs to run: code, runtime, system tools, system libraries, and settings.

Steps to Containerise the Express App

1. Install Docker

Before you can use Docker, you need to install it on your machine. Visit [Docker's official website](#) and follow the instructions for your operating system.

Also check  [this](#) YouTube video to understand docker.

2. Create a `Dockerfile`

A `Dockerfile` is a script with instructions on how to build a Docker image.

First of all, Create a folder called `docker` inside the root of the project.

Then inside it, create a folder called `development`. In this folder we will create our `Dockerfile`.

For our Express app, the `Dockerfile` might look something like:

```
# Use the official Node.js image as our base FROM node:18 # Set the working directory in
side the container WORKDIR /usr/src/app # Copy package.json and package-lock.json first
to leverage Docker cache COPY package*.json ./ # Install app dependencies RUN npm instal
l # Copy the rest of our app's source code into the container COPY . . # Expose the port
the app will run on EXPOSE 5501 # The command to run our app CMD ["npm", "run", "dev"]
```

3. `.dockerignore` File 🚫

To ensure that local node modules and debug logs aren't copied into our Docker image, we'll use a `.dockerignore` file:

```
node_modules npm-debug.log .env
```

4. Building the Docker Image 🏗️

Navigate to the directory containing your `Dockerfile` and run:

```
docker build -t auth-service:dev -f docker/development/Dockerfile .
```

This command will produce a Docker image named `auth-service` with the tag `dev`.

5. Running the Express App in a Docker Container 🚀

Once the image is built, you can run it:

```
docker run --rm -it -v $(pwd):/usr/src/app -v /usr/src/app/node_modules --env-file $(pwd)/.env -p 5501:5501 -e NODE_ENV=development auth-prep:dev
```

i For Powershell users:

use `${PWD}` instead of `$(pwd)`

i For Command prompt (cmd) users:

use `%cd%` instead of `$(pwd)`

Your Express app is now accessible at `http://localhost:5501`.

6. Stopping the Docker Container 🖐

You can stop the running container by using this command:

```
// If container is running in interactive mode. ctr + c // If container is running in detached mode. // List all running container docker ps // Stop the container using container id docker stop <container id>
```

Conclusion 🌟

Docker provides a consistent environment for applications, reducing the "it works on my machine" problem. With your Express app now containerized, you can be confident that it will run the same way everywhere Docker is installed. As you proceed in your MERN stack journey, consider containerizing other parts of the stack for a seamless development and deployment experience.